PitchLog: Personal Soccer Career Tracker

Project blueprint for Abdulmalik Calasow | October 9, 2026

1. Goal and scope

Build a single-user application that records your soccer performances across three leagues and multiple seasons. Calculate goals, assists, hat-tricks, wins, draws, losses, and achievements. Distinctive features: league comparisons and a personal milestone timeline.

Assumptions: you enter records manually; one performance per match; your teams may change between seasons. No live sports API, other player accounts, or AI is needed. Begin locally; use fictional data for a public demo.

2. Recommended software and framework stack

VS Code: edit application code and SQL scripts.

Node.js (supported LTS release meeting Next.js requirements) and npm: run the app and install packages. Next.js documentation currently lists Node.js 20.9 as its minimum [web:120].

Next.js App Router + TypeScript + React: pages, forms, and server-side API routes in one application. Next.js includes TypeScript support [web:120].

PostgreSQL: relational database. pgAdmin: inspect tables and run SQL. Use your course database instead if required, but adapt driver and SQL syntax.

pg (node-postgres): run handwritten, parameterized SQL from server code [web:119]. No ORM initially, so the SQL work is visible.

Plain CSS or CSS Modules: styling. Git + GitHub: version control and project delivery.

Optional later: Vitest for automated tests and a chart library. Python, Express, Docker, and AWS are not required for this version.

3. Architecture

Browser -> Next.js page/form -> server Route Handler -> pg connection pool -> PostgreSQL. Keep database access in server-only modules. Never send database credentials to the browser. Store DATABASE_URL in .env.local and exclude secrets from Git.

4. Database design

leagues: id PK, name UNIQUE. Enter your three leagues; do not hard-code their names into statistics logic.

seasons: id PK, league_id FK, label, start_date, end_date; UNIQUE(league_id, label).

teams: id PK, name. Store your teams and opponents.

matches: id PK, season_id FK, my_team_id FK, opponent_team_id FK, played_on, team_goals, opponent_goals, notes. Store completed matches you played in for version 1.

performances: id PK, match_id FK UNIQUE, goals, assists nullable, minutes nullable, position nullable. You are the only player, so no users or players table is needed.

achievements: id PK, season_id FK, team_id FK nullable, name, type (team_trophy / individual_award / placement), awarded_on nullable, notes. Count trophy records, not all achievement types.

memories (phase 2): id PK, match_id FK, title, note. Use these for personal highlights; derive statistical milestones from records.

Relationships: league 1-to-many seasons; season 1-to-many matches and achievements; match 1-to-1 your performance; team referenced by matches. Use date fields for match dates. Validate season date ranges, nonnegative scores/stats, and different teams. Add a match_id index through its unique constraint; consider an index on (season_id, played_on). Foreign keys preserve valid references [web:101].

5. Statistics definitions

Appearance: one saved match with your performance. Include only completed matches in version 1.

Goals/assists: SUM of recorded values; unknown assists remain NULL, not zero. Show known-assist totals with coverage such as 8 of 10 matches recorded.

Hat-trick matches: COUNT of performances with goals >= 3. Four goals still counts as one hat-trick match.

Win/draw/loss: compare team_goals with opponent_goals. Calculate rather than storing a second result field.

Win percentage: wins / appearances * 100; display N/A with no appearances. Goals per appearance: goals / appearances.

Goals per 90: 90 * goals / minutes, using only rows with known positive minutes for both numerator and denominator. Defer until minutes are reliable.

Trophies: count achievements of type team_trophy. Do not count runner-up placements or individual awards as trophies.

Filters: apply league and season filters before aggregation. Streaks use date order plus match ID as a tie breaker; defer until the core reports are correct.

6. Pages and first-version features

Dashboard: total appearances, goals, known assists, hat-trick matches, wins/draws/losses, trophies; league/season filters.

Match log: sortable match list and add/edit/delete form. Ask for date, league/season, teams, final score, personal goals, optional assists/minutes/position, notes.

League comparison: appearances, goals, assists coverage, goals per appearance, and results for each league. Do not infer league difficulty from these numbers.

Achievements: add trophies, awards, placements; filter by season and type.

Setup: manage leagues, seasons, and teams.

Timeline (phase 2): first goal, first hat-trick, personal highest-scoring match, trophies, and manually written memories. State earliest recorded milestone when historical records are incomplete.

7. API and project organization

Suggested endpoints: GET/POST /api/matches; GET/PATCH/DELETE /api/matches/[id]; GET /api/stats; GET /api/leagues; GET/POST /api/seasons; GET/POST /api/teams; GET/POST /api/achievements. Add league creation/editing when needed.

Suggested folders: src/app for pages and API route handlers; src/components for forms and cards; src/lib/db.ts for the server-only pool; src/lib/queries for SQL access; src/lib/validation for input checks; sql/001_schema.sql, sql/002_seed.sql, sql/reports.sql; tests/ for cases.

Use a database transaction when inserting or updating a match and performance together. Use parameters ($1, $2, ...) rather than concatenating user input into SQL [web:119]. Whitelist allowed sort fields. Return clear validation errors, not raw database exceptions.

8. Build sequence and completion checkpoints

Phase 1 - Design: write field definitions and draw the ER diagram. Done when every statistic has a defined input and calculation.

Phase 2 - Database: create tables, keys, checks, sample records, and reporting queries. Done when reports work directly in pgAdmin without a UI.

Phase 3 - App foundation: create Next.js project, connect PostgreSQL, and show seeded matches. Done when a server route returns real database records.

Phase 4 - Match management: implement create/edit/delete and validation. Done when changes survive reload and failures cannot leave half-saved records.

Phase 5 - Dashboard: add filtered totals and league comparison. Done when each displayed total matches independently verified SQL results.

Phase 6 - Achievements and timeline: implement achievements first, milestone timeline second. Done when timeline correctly handles incomplete history.

Phase 7 - Delivery: tests, README, ER diagram, screenshots, and demo recording. Deploy only after deciding how to protect personal records.

9. First commands

Install VS Code, Git, Node.js, PostgreSQL, and pgAdmin. Create a local database named pitchlog. In a terminal run:

npx create-next-app@latest pitchlog

Choose TypeScript, ESLint, App Router, and src directory. Styling choice is optional.

cd pitchlog

npm install pg

npm install -D @types/pg

Set DATABASE_URL in .env.local; use your local database credentials.

Run schema and seed scripts in pgAdmin, then npm run dev.

10. Test dataset and expected results

Create fictional completed matches with (goals, assists, result): (3,1,win), (4,0,win), (0,NULL,draw), (1,2,loss). Expected: 4 appearances, 8 goals, 3 known assists with 3/4 coverage, 2 hat-trick matches, 2 wins, 1 draw, 1 loss, 50% win rate. Spread records across three leagues and check each filter separately.

Also test: no matches; missing assists; negative scores rejected; invalid season rejected; teams cannot be identical; delete updates totals; edit a three-goal match to two goals removes its hat-trick; duplicate performance rejected; transaction rollback on failure. Keep fictional seed data separate from personal data.

11. Deployment and privacy

Version 1 can remain local with screenshots and a recorded walkthrough. For a public demo, use fictional data and disable write routes server-side. Hiding buttons is not protection. If storing real records online, add authenticated write access before release; keep secrets private and avoid publishing other people's personal information. Hosting and authentication choices are separate later decisions, not prerequisites.

12. Resume-ready deliverables

Deliver a working repository, schema/seed/report scripts, ER diagram, setup guide, tests with expected outputs, screenshots, and a brief design explanation. List only features you actually completed. Suggested future bullet: Designed a PostgreSQL-backed soccer tracker and wrote SQL joins and aggregate reports to compare personal performances across three leagues and multiple seasons.

13. Decisions to confirm before coding

Confirm your three league names; whether they use seasons; whether you change teams; whether you know historical assists/minutes; and whether your SQL class requires a particular database. These do not block the blueprint, but they may change seed data and implementation choices.

Source notes

[web:120] Official Next.js installation documentation: framework setup, Node.js minimum, TypeScript support. [web:119] node-postgres queries documentation: parameterized SQL. [web:101] PostgreSQL tutorial: foreign keys. Remaining architecture, scope, and milestones are recommendations tailored to this project.